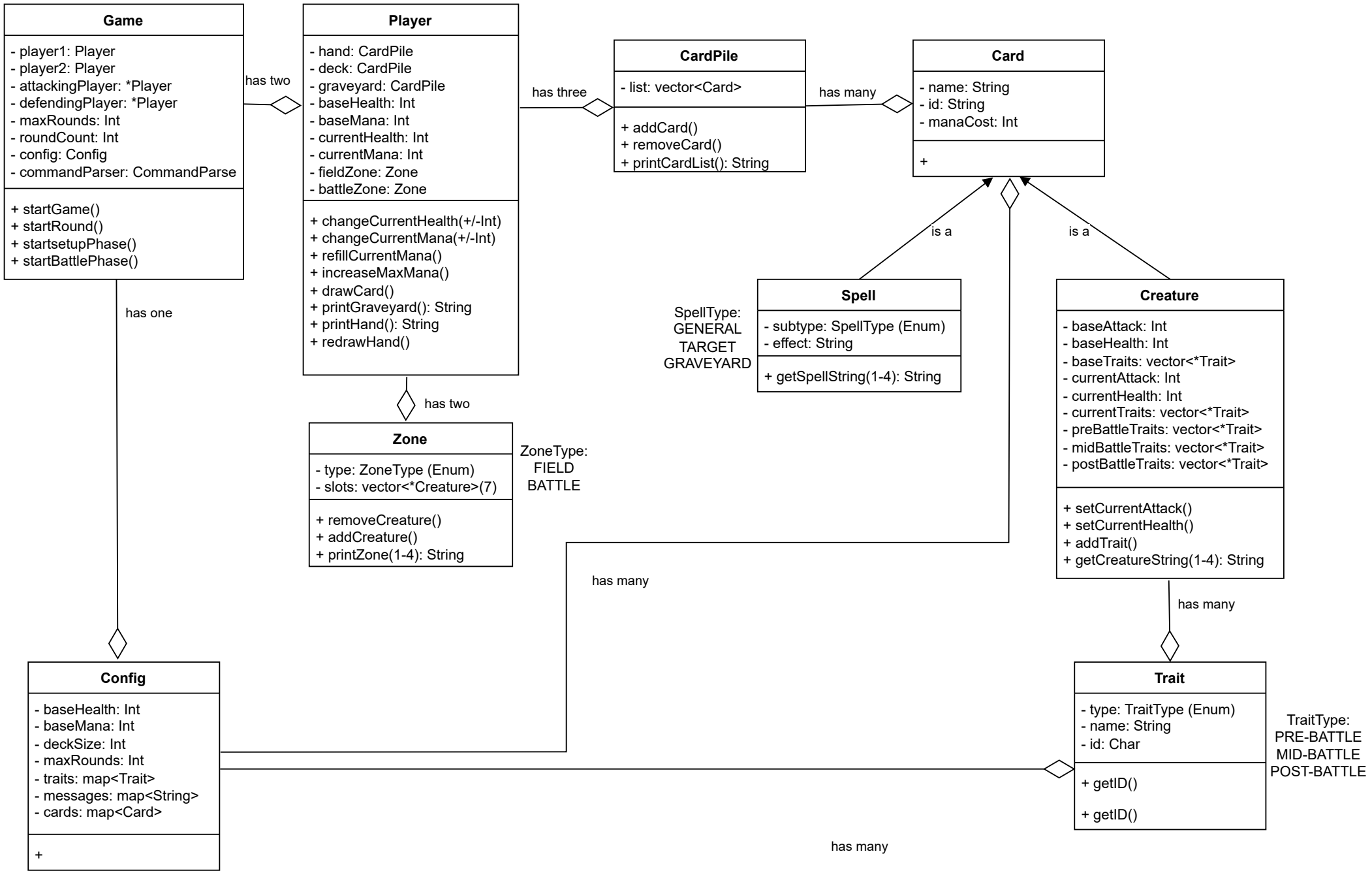
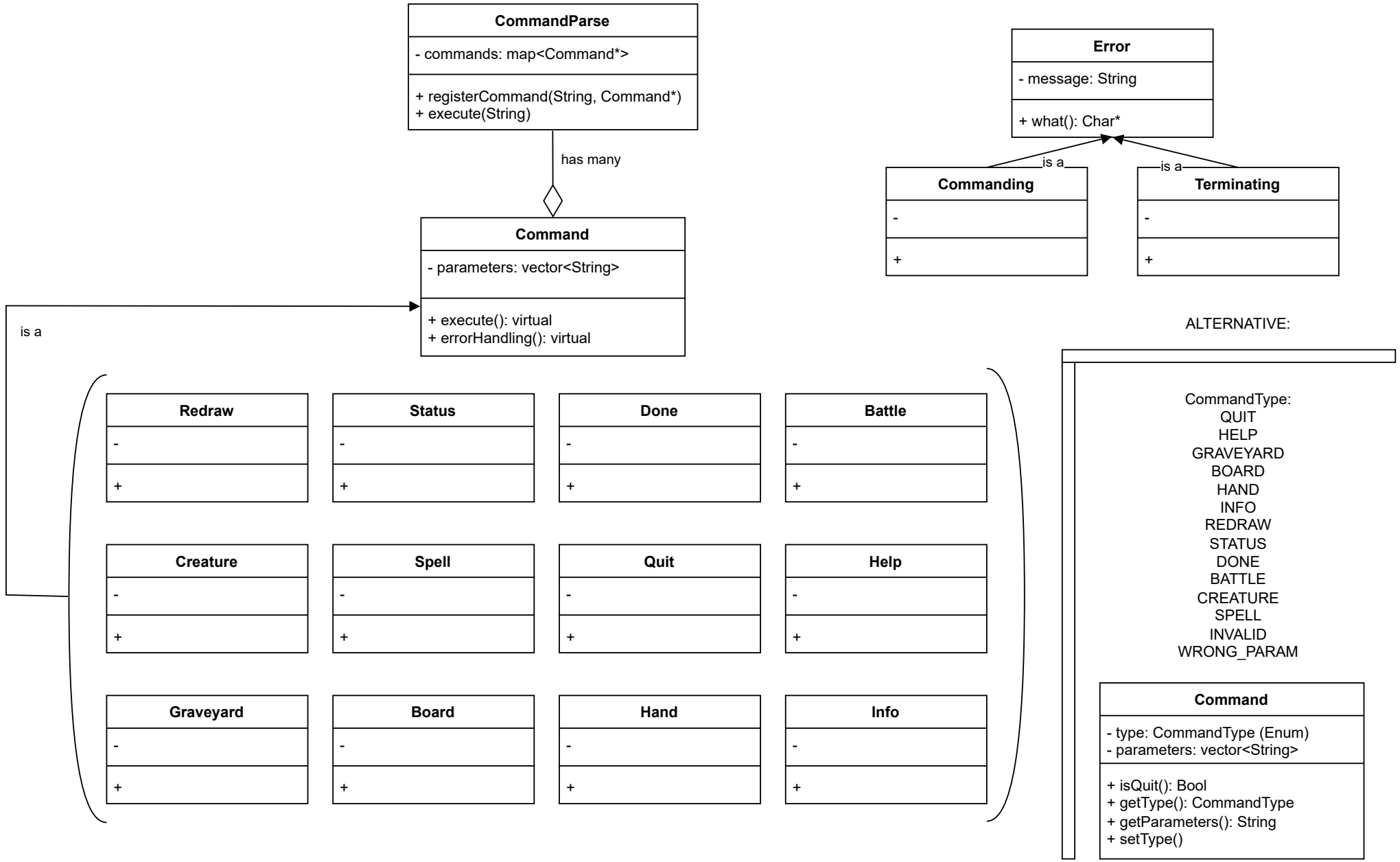

Game
- player1: Player
- player2: Player
- attackingPlayer: *Player
- defendingPlayer: *Player
- maxRounds: Int
- roundCount: Int
- config: Config
- commandParser: CommandParse
+ startGame()
+ startRound()
+ startsetupPhase()
+ startBattlePhase()
has two
Player
- hand: CardPile
- deck: CardPile
- graveyard: CardPile
- baseHealth: Int
- baseMana: Int
- currentHealth: Int
- currentMana: Int
- fieldZone: Zone
- battleZone: Zone
+ changeCurrentHealth(+/-Int)
+ changeCurrentMana(+/-Int)
+ refillCurrentMana()
+ increaseMaxMana()
+ drawCard()
+ printGraveyard(): String
+ printHand(): String
+ redrawHand()
has three
CardPile
- list: vector<Card>
+ addCard()
+ removeCard()
+ printCardList(): String
has many
Card
- name: String
- id: String
- manaCost: Int
+
has one
has two
Zone
- type: ZoneType (Enum)
- slots: vector<*Creature>(7)
+ removeCreature()
+ addCreature()
+ printZone(1-4): String
ZoneType:
FIELD
BATTLE
is a
Spell
- subtype: SpellType (Enum)
- effect: String
+ getSpellString(1-4): String
SpellType:
GENERAL
TARGET
GRAVEYARD
is a
Creature
- baseAttack: Int
- baseHealth: Int
- baseTraits: vector<*Trait>
- currentAttack: Int
- currentHealth: Int
- currentTraits: vector<*Trait>
- preBattleTraits: vector<*Trait>
- midBattleTraits: vector<*Trait>
- postBattleTraits: vector<*Trait>
+ setCurrentAttack()
+ setCurrentHealth()
+ addTrait()
+ getCreatureString(1-4): String
has many
has many
Config
- baseHealth: Int
- baseMana: Int
- deckSize: Int
- maxRounds: Int
- traits: map<Trait>
- messages: map<String>
- cards: map<Card>
+
has many
Trait
- type: TraitType (Enum)
- name: String
- id: Char
+ getID()
+ getID()
TraitType:
PRE-BATTLE
MID-BATTLE
POST-BATTLE


CommandParse
- commands: map<Command*>
+ registerCommand(String, Command*)
+ execute(String)
has many
Command
- parameters: vector<String>
+ execute(): virtual
+ errorHandling(): virtual
is a
Error
- message: String
+ what(): Char*
is a
is a
Commanding
-
+
Terminating
-
+
Redraw
Status
Done
Battle
Creature
Spell
Quit
Help
Graveyard
Board
Hand
Info
ALTERNATIVE:
CommandType:
QUIT
HELP
GRAVEYARD
BOARD
HAND
INFO
REDRAW
STATUS
DONE
BATTLE
CREATURE
SPELL
INVALID
WRONG_PARAM
Command
- type: CommandType (Enum)
- parameters: vector<String>
+ isQuit(): Bool
+ getType(): CommandType
+ getParameters(): String
+ setType()

The **Game** Class is the main class of the Program. It deals with initializing and executing the game and handling the different rounds. Next to Integers used for counting, you will also find **Player** Classes and the **Config** Class here. It also contains a **Command Parser** used to execute commands.

The **Config** Class deals with all of the settings specified by the configuration documents provided and manually written. It stores default values and libraries of **Cards**, messages and **Traits**.

> For the libraries of cards and traits, we will include a manually written config file that contains these and will be mapped for later use in the program.

The **Player** Class handles a player's interaction with cards as well as their side of the game field. This includes **Card Piles** such as hand cards, deck and graveyard, but also their two **Zones** of the board.

The **Zone** Class handles a zone of 7 slots that can be found on the board. The type of zone can be specified.

The **Card Pile** Class is for lists of **Cards** and allows for better handling of these lists.

The **Card** Class specifies default values that every type of card in the game has, like name, ID and mana cost. It can be split into two different types of cards: **Spell Cards** and **Creature Cards** which both also deal with the printing of themselves.

> **Spell Cards** can be given a spell type.
>
> **Creature Cards** have base stats and current stats. These stats include health, attack and also **Traits**, which are divided into pre-battle, mid-battle and post-battle traits.

The **Trait** Class allows for specification of the traits given to **Creature Cards**.

The **Error** Class allows for dealings with errors. For organizational purposes, it can be split into **Commanding Errors** and **Terminating Errors**.

The **Command Parser** Class stores a map of the string used to call the command and the corresponding **Command** object.

The **Command** Class deals with inputted user commands and handles the execution of these commands. For this execution, we intend to have multiple subclasses for each command in the game; **Redraw**, **Status**, **Done**, **Battle**, **Creature**, **Spell**, **Quit**, **Help**, **Graveyard**, **Board**, **Hand** and **Info**.